

Accurate Prediction of Detailed Routing Congestion using Supervised Data Learning

Zhongdong Qi
Computer Science Department
Tsinghua University
Beijing, China
zhongdongqi@gmail.com

Yici Cai
Computer Science Department
Tsinghua University
Beijing, China
caiyc@mail.tsinghua.edu.cn

Qiang Zhou
Computer Science Department
Tsinghua University
Beijing, China
zhouqiang@mail.tsinghua.edu.cn

Abstract—Routing congestion model is of great importance in design stages of modern physical synthesis, e.g. global routing and routability estimation during placement. As the technology node becomes smaller, routing congestion is more difficult to estimate during design stages ahead of detailed routing. In this paper, we propose a framework using nonparametric regression technique in machine learning to construct routing congestion model. The constructed model can capture multiple factors and enables direct prediction of detailed routing congestion with high accuracy. By using this model in global routing, significant reduction of design rule violations and detailed routing runtime can be achieved compared with the model in previous work, with small overhead in global routing runtime and memory usage.¹

I. INTRODUCTION

In modern physical synthesis of VLSI circuits, routability has raised as a primary concern, even more important than timing [2] [9]. Global routing (GR) is one of the most important design stages in physical synthesis flow, which has two major functionalities. A global router plans routing trees on a coarse grid to minimize congestion and wirelength (as well as other design objectives) and feeds the results to a detailed router to perform detailed routing (DR). A “light-mode” global router also serves as a congestion estimator during routability-driven placement to guide the placer producing routable solutions [10] [8]. During global routing, a routing congestion model is used to estimate detailed routing congestion according to global routes such as wire segments and vias.

As the technology advances, there has been a considerable gap between routing congestion model used in global routing and real congestion in detailed routing. To address lithography, chemical mechanical polishing, and yield issues, design rules involved in each technology node keep increasing in number and become more and more complex. There have been different kinds of design rules, such as diagonal width, minimum step, minimum area, minimum enclosed area, notch, minimum cut, and width-length-dependent spacing. The design rules forbid certain pattern and relative positions of layout geometries, and largely affect routing resource allocation in detailed routing. Further, the technology scaling leads to denser pins on chip, which makes pin access paths consume significant routing resources on lower routing layers. What’s more, increasing number of routing layers over successive technology

nodes introduce more vias into interconnections, which are difficult to measure in traditional routing congestion model.

To reduce this gap, there have been several pioneering work attempted to improve the estimation accuracy of the routing congestion model. A via capacity and via overflow concept was proposed by Hsu *et al.* to constrain via number in global routing [6]. Wei *et al.* [16] used Steiner trees and pin factor to estimate resources consumption contributed by local pin access paths. A vertex capacity concept was employed by Shojaei *et al.* [13] to measure local routing resources. Vertex demand was used to measure routing resources used by local nets. Zhang and Chu [18] employed Voronoi Diagram to measure pin distribution, and assigned weights to global routing graph edge. The capacity of global routing graph edges were iteratively updated according to detailed routing results to reflect detailed routing difficulty in different routing regions. Recently, Qi *et al.* [11] used pass-through capacity and demand to measure local congestion contributed by vias and local connections, with explicitly modeling of major design rules.

The above work improved the accuracy of routing congestion model used in global routing. There are still some limits with these models. 1) It is difficult to explicitly modeling every sophisticated design rule explicitly. Facing increasing number of design rules, it is more difficult to do so in smaller technology nodes. 2) The resource consumption and congestion contributed by local pin access paths are difficult to estimate. The previous work only used simple tree structure or pin count factor to measure the resource consumption, which did not capture other important factors. 3) Factors contributing to routing congestion were assumed to be independent. However, they may interact with each other. For example, the via enclosure number and the pin distribution may interact, and their impact on routing congestion varies from configurations, which does not obey a simple superposition relation. Their co-effect on resource consumption is difficult to measure using previous approaches.

Due to above limitations, congestion estimated in global routing is not correlated to real detailed routing congestion well enough. An example from recent work [11] is shown in Fig. 1. In the figure, the left part is a partial congestion map showing the estimated congestion hotspots on gcell grid. Illustrated by the legend on bottom left, the gcells with overflow are highlighted in the congestion map. The right part is design rule violation markers of detailed routing results corresponding to the layout region shown on the left. We can see that some

¹This work was supported by Natural Science Foundation of China (NSFC) No.61274031.

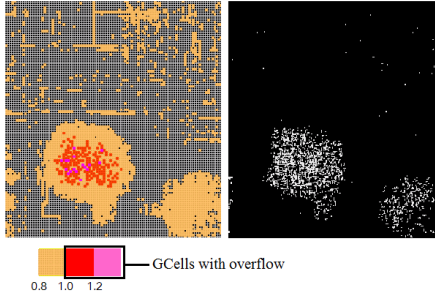


Fig. 1. The correlation between congestion heatmap in global routing and design rule violation markers in detailed routing

regions regarded as not congested in global routing actually have a lot of design rule violations in detailed routing.

To overcome these limitations, we adopt the machine learning approach to construct accurate congestion model which can capture multiple factors. Machine learning method has been used to address a number of important problems in design automation of VLSI. In recent years, it was used to model NoC router power, delay and area by Jeong *et al.* [7]. Ghaida *et al.* used the machine learning approach to predict design characteristics in up-coming technology nodes [5]. It was also employed to solve lithography hotspot detection problem and structured design placement [3] [15]. For congestion modeling problem, by utilizing the routing data, the learning algorithm can capture the relation between congestion contributing factors and detailed routing congestion, with interaction among the factors which cannot be easily captured by manual modeling.

Our contributions are as follows:

- We propose a new machine-learning-based method to construct routing congestion model without explicitly modeling every sophisticated design rule. The method utilizes real detailed routed design data to more accurately model routing congestion. The model captures various factors contributing to routing congestion as well as interaction between different factors.
- We employ a novel congestion mapping function which directly maps data in global routing to design rule violations in detailed routing results. This helps constructing a congestion model to enhance convergence of global routing and detailed routing.
- We develop effective critical feature extraction method for congestion model training, which enables high accuracy of constructed models.
- We integrate the constructed models into global routing, and show significant improvement in detailed routing runtime and solution quality compared with the model in previous work.

The remainder of the paper is organized as follows. Section II gives the problem definition of routing congestion modeling. Section III proposes the methodology of constructing congestion model using machine learning. Section IV describes the usage of constructed congestion model in global routing. Experimental results are presented in Section V followed by conclusions.

II. PROBLEM DEFINITION

In global routing, the routing region is typically tessellated into rectangular sub-regions called global routing cells (a.k.a. gcells). A multi-layer grid graph is constructed based on the tessellation. The global router constructs routing trees coarsely on the grid graph for interconnections (nets) guided by estimated congestion in gcells. The results of global routing are feed into a detailed router. The detailed router searches paths in a small region (e.g. a gcell) guided by global routes. It attempts to avoid design rule violations during path search and constructs real geometries for wires and vias.

In the global routing grid graph, each vertex represents a gcell on a certain layer, while each edge represents the region between two neighboring gcells. The routing congestion in each vertex or each edge can be estimated according to global routes (e.g. wire segments, vias) in it. A global router can either use the edge congestion (e.g. [12]) or the vertex congestion or both (e.g. [11]) to guide the path search.

Given a gcell with global routes information, e.g. number of vias and wire segments in the gcell, what we seek for is a method of accurately estimating the detailed routing congestion for the gcell. Fig. 2 shows an detailed routed gcell with a design rule violation (spacing violation marked by a highlighted rectangle with a cross). The gcell has 11 wire segments and 8 via enclosures, two of which are stacked via enclosures. Only the numbers but not exact locations are known in global routing stage.

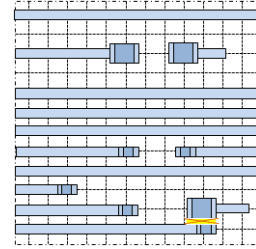


Fig. 2. A detailed routed gcell with a design rule violation marker

In each gcell, the routing congestion is contributed and affected by multiple factors, including pin geometries and distributions, the number of wire segments and vias, as well as design rules.

We define the problem of routing congestion modeling: given technology design rules in a certain technology node, using the global routes and layout information (e.g. blockages, pin geometries and distributions) in each gcell, construct a model which estimates real detailed routing congestion.

III. METHODOLOGY

To construct a congestion model which directly estimates detailed routing congestion, we employ a new mapping function from global routes and layout data to detailed routing congestion. Then a machine learning approach is used to learn the correlation of objects seen in global routing and congestion in detailed routing.

How design rules affecting the resource consumption and congestion in detailed routing is learned by the learning

algorithm, so explicitly modeling of each sophisticated design rule is not required. Since the technology library used by different technology nodes varies a lot, it is very difficult and not necessary to construct one model which can be used for all technology nodes and all designs. Our machine-learning-based framework can be adapted to different technology nodes by running the learning flow.

A. A New Congestion Mapping Function

The detailed routing congestion can be measured by two metrics:

1. The utilization of routing resources, i.e. the ratio of routing resources consumed by detailed routing paths.
2. The number of design rule checking (DRC) violations, including layout versus schematic (LVS) violations, e.g. opens and shorts.

Traditional congestion model used in global routing and congestion estimation during placement has a mapping function of the form:

$$f : \mathbf{N} \rightarrow \mathbf{R}$$

where the wire segment number in global routing maps to the utilization of resources in detailed routing. The set of the wire segment number and the utilization is represented by $\mathbf{N}$ and $\mathbf{R}$, respectively.

To directly model the detailed routing congestion and the detailed router behavior, we can use a new mapping function:

$$f : (\mathbf{N} \times \mathbf{N} \times \Phi) \rightarrow (\mathbf{N}, \mathbf{R})$$

in which the segment number, the via number, pin distribution data (whose space represented by Φ) are mapped to the number of DRC violations and the utilization of detailed routing resources. The function input may include more data.

In the process of model construction, the DRC violation count and the utilization of routing resources in each gcell are treated as modeling targets.

B. Overall Flow of Modeling Method

Since the behavior of a detailed router is hard to describe and formulate, data of real routed designs are of great value in modeling detailed routing congestion.

The overall modeling flow comprises of data collection with critical feature extraction, model training, as well as model validation and test. The constructed model is finally integrated into global routing.

As depicted in Fig. 3, data of layout and global routes are first collected from the global routing stage. Data of detailed routing congestion including the utilization and the DRC violations are collected from the detailed routing stage. These form the labeled data for training the model. Then a machine learning algorithm is performed to construct routing congestion model. The constructed model can be integrated into global routing if it has good prediction performance.

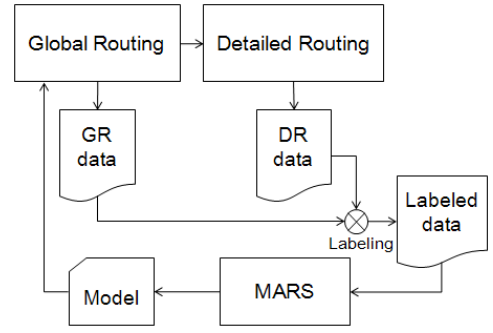


Fig. 3. Overall data flow

C. Critical Feature Extraction

The detailed routing congestion is contributed by various factors. To construct an accurate model using machine learning, critical features are explored.

Local pin access paths are a crucial issue for global routing and detailed routing. Pin access paths contribute to routing congestion on low metal layers, which is hard to model ahead of the detailed routing stage. Pins with different geometries and distributions have very different access difficulty and access paths, which consume different amount of routing resources and cause different amount of design rule violations.

Fig. 4 shows two examples similar to those presented in ISPD 2014 detailed-routing driven placement contest and benchmarks [17]. In the figure, there are two gcells (with height equal to standard cell height) with boundaries of thin solid lines. The left gcell has 5 pins to connect during routing, while the right one has 3. Each pin in two gcells has different shape, with other pins and blockages in its neighborhood. The pin count, pin shape size, and pin neighboring information all affect the pin access difficulty and amount of routing resource consumed by pin access paths. Pins in the right gcell are easier to be connected and would consume less routing resources on lower layers above the pin layer, since the number of them is smaller, each of them has bigger shape size, and there are more free room surrounding each pin for accessing. It is also possible that they cause less DRC violations than those in the left gcell.

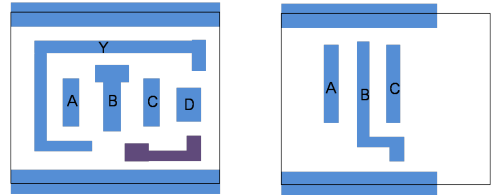


Fig. 4. Two gcells with different pin count, pin shape size, and pin distribution

As a result, the pin count, the pin shape size and the pin neighboring information are selected as critical features related to local pin access paths.

The pin information data extraction are also applied to macro block pins and random-logic-macro (RLM) pins introduced in modern hierarchical System-on-Chip designs [14]. The pins of macro blocks, e.g. memory and datapath blocks, are typically on top of these blocks (e.g. on Metal4 for a 9

metal layer stack in 45nm technology node). The RLM pins reside on the upper-level metal layers (e.g. Metal5). Accessing these pins consumes extra routing resources.

To calculate pin information data, the pin count is the total number of pins in a gcell. The pin shape size is computed using the area of pin shape. It can be measured by the number of detailed routing grid points which covered by pin shape. But in smaller technology nodes, a pin can be off-grid, where the pin shape's area actually matters.

The pin neighboring metric is calculated as follows. For a target pin, first, all the other pin figures and blockages are bloated by minimum spacing on the layer. Then a window constructed by bloating the target pin shape by one routing pitch is used to calculate the ratio of blocked area inside the window to the entire window area. All these operations are polygon-based. This is illustrated in Fig. 5, which is corresponding to the layout shown in Fig. 4. In the figure, the target pin is pin A. The rectangles with dashed line boundaries are the calculation windows. As shown in the figure, pin neighboring metric for pin A in the left gcell is bigger than the one in the right gcell.

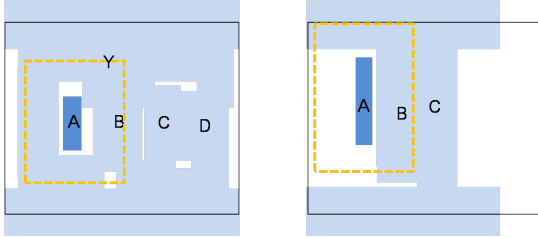


Fig. 5. Demonstration of pin neighboring metric calculation

After calculating metrics of pin shape size and pin neighboring for each pin, the statistical measurement of these data are used as feature values. The average, standard deviation, and top $X\%$ mean value are employed. X are 20 and 50 in our implementation.

Global routes are also contributors to detailed routing congestion. For a gcell, the critical features of global routes include the number of side-connecting segments, the number of segments through the gcell, as well as the number of stacked vias and the number of non-stacked vias. The capacity of a gcell is also a feature. It is computed according to partial or full blockage in the gcell.

From the above, each labeled data point in the data set has attributes of information of pins on the layer not higher than the gcell, the capacity, and attributes of global routes information including various kinds of segments number and via number. These critical features are extracted from layout and global routing results. It is further labeled by utilization of resources and design rule violation count extracted from detailed routing results. This is shown in Fig. 3.

D. Model Training by Nonparametric Regression

Since we have labeled data, supervised learning method is chosen to train the model. There are various supervised learning algorithms which have been successfully applied in model construction problems. For routing congestion modeling, we use multivariate adaptive regression splines (MARS).

MARS is a non-parametric regression technique for flexible regression modeling of high dimensional data [4]. It automatically models non-linearities and interactions between variables. It is powerful in modeling relationships that are nearly additive or involve interactions in at most a few variables.

MARS builds models of the form

$$\hat{f}(x) = c_0 + \sum_{i=1}^N c_i \prod_{j=1}^{d_i} h_{ij}(x_{ij}).$$

The input is x , $\hat{f}(x)$ is its prediction function. The model is a weighted sum of N products of basis function $h_{ij}(x)$. Each product has a degree d_i representing the number of terms in it. Each c_i is a constant coefficient. Each basis function $h_{ij}(x)$ has one the two forms: a constant 1; a hinge function $\max(0, x - \text{const})$ or $\max(0, \text{const} - x)$.

As a result, the reasons of choosing MARS for congestion modeling are as follows. 1) It captures non-linearity of functions. 2) Interactions between variables can be easily captured. 3) Time-consuming parameters tuning is not needed. 4) It is easy to integrate the constructed models to global routing and congestion estimation in physical synthesis.

The training data are feed into MARS, and the routing congestion model expressions are got from MARS output. To ensure the generalization of model to unseen data, the prediction model is validated using validation data set and further tested by large-scale congestion data.

IV. MODEL INTEGRATION IN GR

The constructed routing congestion model can be integrated into global routing if it has good prediction performance during validation and test. Because the model uses several input features which are critical for modeling detailed routing congestion, the global router needs to prepare these data. The data include pin information, global routes count, and available gcell area.

A. Integrated GR Flow

A procedure of pin information data calculation is added in the beginning of global routing. The pin count, pin shape size, and the pin neighboring information are calculated and stored. In the path search and routing tree construction procedure of global routing, the number of side-connecting segments, the number of segments through the gcell, the number of stacked vias, and the number of non-stacked vias in each gcell is updated and stored. The utilization and design rule violation count are calculated using new congestion model expressions to predict detailed routing congestion.

B. From Model to Path Search Cost

Generally, global routing searches paths and constructs routing trees according to gcell congestion. The ratio of a demand value and a capacity value $\text{demand}/\text{capacity}$ is typically used to measure the routing congestion, and a cost value is computed according to it.

Using constructed congestion model with new congestion mapping function, the routing congestion value r can be

formed by

$$r = \begin{cases} \text{utilization}, & \text{if } drc = 0 \\ 1 + \frac{drc}{N_m}, & \text{else} \end{cases}$$

where *utilization* is the resource utilization rate while *drc* the DRC violation count predicted by the model. N_m is a constant design rule violation count defined by the model user to measure the level of congestion.

V. EXPERIMENTAL RESULTS

A. Experiment Setup

The data set was generated from full-flow routing using DAC 2012 routability-driven placement benchmarks. The nine DAC 2012 routability-driven placement benchmark designs contains several challenges in modern technology [14]. These designs have massive hard macros, varying metal layers width and spacing, macro pins and RLM pins on upper-level metal layers. The placement solutions were produced by placers in DAC 2012 routability-driven placement contest. 45nm design rules were added to the designs. Pin figures and obstacles in standard cells and macro blocks were constructed. The gcell height was equal to placement row site height. So the gcell grid had more gcells than the tessellation used in original benchmarks.

The full-flow routing comprised of a global routing stage by an academic global router [11], and a detailed routing stage by a commercial detailed router which supported sophisticated design rules. The statistics of congestion data generated from benchmarks are shown in Table I. In the table, “*L*” is the routing layer number, “*Nets*” is the total number of nets, “*GCell Grid*” is the dimension of gcell grid. “*Data Size*” represents the number of data points on each routing layer.

TABLE I. STATISTICS OF BENCHMARKS AND LABELED DATA.

Circuit	<i>L</i>	Nets	GCell Grid	Data Size
superblue2	9	990899	2738 x 3960	32525440
superblue3	9	898001	2845 x 1845	15747075
superblue6	9	1006629	2308 x 2200	15232800
superblue7	9	1340418	1775 x 3169	16874925
superblue9	9	833808	1515 x 2534	11517030
superblue11	9	935731	2244 x 3903	26274996
superblue12	9	1293436	1579 x 2303	10909311
superblue14	9	619815	1444 x 2103	9110196
superblue19	9	511685	1143 x 2303	7896987

We used data generated from six out of nine designs as test data set. These designs for test data were randomly chosen. Data from remaining three designs (superblue9, superblue11 and superblue19 in our experiments) were used as training set and validation set. They were randomly divided into 10 equally-sized subsets, among which one was used as training set and the other nine together as validation set. Because our data set was large (multi-million data points per circuit), the training data set still had 1.5 million data points, we used data splitting and validation instead of *k*-fold cross-validation.

The data processing and learning was implemented in Python. MARS regressor we used was open source Earth package [1]. After we got the new congestion model, it was integrated into the global router to investigate its impact on routing flow. The global router was written in C++.

The routing flow was run on a Linux server with 8-core 2.40GHz CPU and 24GB memory. Data processing and model training were performed on a machine with 3.40GHz CPU and 4GB memory.

B. Model Validation and Quality Evaluation

The model was trained using the training data set. Since there were redundant data points in the data set, the training data set was preprocessed to remove redundancy. This decreased the number of training data points, which in turn reduced the training runtime. The training runtime is shown in Table II, in unit of seconds. Training a model of utilization lasted less than 20 seconds, while the one of design rule violation count lasted less than 2 minutes.

During validation and model selection, we observed that the model’s quality highly depended on the diversity of data points in the training data set. If the model was trained only by data with global routing of high-effort congestion elimination, the model performed poor on data dumped from intermediate stage of global routing, where each data point had much more global route objects in its corresponding gcell. Therefore, data from different stages of global routing were used to cover possible range of features. In another word, different levels congestion reduction effort in global routing were used to obtain data points with good diversity.

Another factor for model selection was the maximum term degree for MARS learning algorithm. Higher degree enabled multiplication of more features, which led to more complex model. Different degrees scaling from 1 to 4 were tried and validated. We found that the best degree for utilization was 1 and for design rule violation 2. Degrees higher than the best value caused over-fitting, where the model had smaller error on training data but larger error on validation data. From model selection process, the best model for utilization and DRC violation count on each layer was selected, respectively.

The validation and test results of model quality are shown in Table II, Table III and Table IV. The quality measurement is mean absolute error (MAE). The utilization and design rule violation model was constructed in a per-layer manner to reflect the requirement of varying design rules on each layer. Because Metal1 was typically not used in global routing, modeling for Metal1 was not needed. To show the quality of models on different designs, the test data from six designs were not merged into a single one. Each design had test data with the size shown in Table I per routing layer.

TABLE II. TRAINING RUNTIME AND MAE ON VALIDATION DATA SET.

Layer	Training Runtime		MAE	
	Utilization	DRC	Utilization	DRC
Metal2	18.73	74.80	0.0167	0.2610
Metal3	16.96	70.69	0.0151	0.1269
Metal4	11.79	24.08	0.0196	0.1468
Metal5	9.50	18.90	0.0232	0.0575
Metal6	8.74	16.40	0.0300	0.0239
Metal7	5.93	14.19	0.0079	0.0049
Metal8	4.25	11.45	0.0197	0.0003
Metal9	2.85	9.16	0.0068	0.0015

From the Table II, we can see that validation error was within 0.03 for utilization model, while within 0.3 for design rule violation count model. The utilization was a real number between 0 and 1, while the DRC violation count a natural

number. So the constructed models was accurate enough to be used in future data.

Table III and Table IV show the performance of the model generalized to unseen data and designs. The errors were stable over data from different designs, which were in a small range. From these results, the models had high accuracy and could be used in predicting detailed routing congestion in global routing stage.

From the constructed model expressions, we saw that interaction between different factors was captured. What's more, the coefficients in the model reflected the design rules' influence on detailed routing congestion, which were constructed by the learning algorithm. This made explicitly modeling of each sophisticated design rule not required.

The importance of each feature variables can also be achieved from MARS for model analysis and knowledge extraction. For example, the feature importance indices of the a model for Metal2 DRC violation count are listed in Table V. This identifies important features affecting the model prediction. Utilizing the importance indices, unimportant features can be eliminated from feature data preparation and storage during applications of the models, to reduce runtime and memory.

TABLE V. FEATURE VARIABLE IMPORTANCE IN A DESIGN RULE VIOLATION COUNT MODEL.

Feature	Importance Index
MIPinCount	30.0
throughSegCnt	29.0
nonStackViaCnt	28.0
leftSegCnt	26.0
rightSegCnt	25.0
stackViaCnt	23.0
MIPinBlockACE50	17.0
capacity	14.0
MIPinBlockAvg	11.0

C. Impact on GR-DR Routing Flow

To show the model's impact on global routing and detailed routing, we integrated the model into a global router, and run full-flow routing on DAC 2012 routability-driven placement benchmark designs. The DR results were checked by a commercial design rule checker. Although the model was trained by data from three designs among these designs, it was still meaningful to validate the model on these three designs. It was because in GR, variables with values different from the data set used in model learning were generated.

We compare results of the routing flow using congestion model in previous work [11] and the one using the new model proposed in this paper. The results are shown in Table VI. In the table, "OF" represents total edge overflow and pass-through overflow, which were congestion metrics used in work [11]. "WL" is the total wirelength in unit of 10^{10} nm, while "Via" the total via count in unit of 10^6 , "DRC" the number of DRC violations (including LVS violations) in DR. "DP" represents total DRC violation count predicted by the new model in GR, by aggregating the predicted count in all gcells. Because the global routing using proposed model calculates the utilization and DRC violation number to measure relative congestion, we show predicted DRC violation number instead of overflow. "DRC" and "DP" are in unit of 10^4 . "CPU" and "Mem" represent runtime and memory usage in unit of minutes and gigabytes, respectively.

In Table VI, we can see that the global router using the new model consumed more memory and runtime. The increase of memory was due to store the feature variables in each gcell. The increase of runtime was due to 1) calculating pin information related feature variables in the beginning of GR, 2) calculating utilization and design rule violation count using the new model, and 3) reducing congestion in congested regions which were regarded as congestion-free by the model in [11]. To reduce the congestion, some detours were used and some wire segments were pushed to upper layers, which in turn increased wirelength and via count by 1% and 2% compared with the global router using the model in [11], respectively. After DR, the routing flow using solutions of GR with the new model achieved 34% fewer design rule violations, as well as 2% shorter wirelength and 3% smaller via count. The runtime of DR also decreased by 37%. The overhead of DR runtime was much smaller compared with the reduction of DR runtime, which meant a significant reduction of total routing runtime. The DRC violation count predicted in GR was also close to the number of actual DRC violations in DR, which showed the predictive accuracy of the proposed model.

Since the global router used in the routing flow had a post-processing stage for optimizing overflow, which used congestion model in work [11] and did not support the proposed congestion model for technical reasons. As a result, in routing flow using proposed congestion model, the post-processing stage would degrade the congestion optimized by previous stages in GR using proposed congestion model (because proposed model is more accurate than the model in [11]). To fully explore the impact of proposed model, we disabled the post-processing stage of GR in both flows using different congestion models. The routing results are shown in Table VII. We can see that DRC violations were decreased by 37%, and via count in DR results was reduced by 13% on average, by using proposed model. This further shows the benefits of using proposed congestion model in GR.

Compared with the model in work [11], the proposed congestion model captures local pin access issue more effectively, using critical features related to pins. It also models vias more accurately using learning algorithm, which is heuristical in the model of work [11]. Further more, it captures interactions between different factors contributing to routing congestion. All of these make the proposed model more accurate than the one in work [11]. Guided by the model, GR can produce detailed-routing-friendly results to provide space and opportunity for DR to deliver better performance and results. Using the new model can facilitate tighter convergence in routing flow.

VI. CONCLUSIONS

In this paper, we propose a machine learning based framework which constructs routing congestion models. The constructed model can capture multiple factors contributing to routing congestion and enables direct prediction of detailed routing results with high accuracy. By using the model in global routing, significant reduction of DRC violations and detailed routing runtime can be achieved compared with the model in previous work.

REFERENCES

- [1] Earth: An implementation of MARS algorithm for Orange. <http://pythonhosted.org/orangecontrib.earth/>.

TABLE III. ACCURACY (MAE) OF UTILIZATION MODEL ON TEST DATA SET.

Test Design	Metal2	Metal3	Metal4	Metal5	Metal6	Metal7	Metal8	Metal9
superblue2	0.0173	0.0155	0.0200	0.0253	0.0342	0.0128	0.0316	0.0113
superblue3	0.0236	0.0231	0.0302	0.0368	0.0380	0.0220	0.0250	0.0247
superblue6	0.0246	0.0238	0.0312	0.0384	0.0445	0.0175	0.0329	0.0142
superblue7	0.0328	0.0311	0.0417	0.0453	0.0450	0.0197	0.0272	0.0127
superblue12	0.0390	0.0388	0.0508	0.0568	0.0541	0.0304	0.0387	0.0221
superblue14	0.0287	0.0276	0.0346	0.0406	0.0472	0.0198	0.0361	0.0115

TABLE IV. ACCURACY (MAE) OF DESIGN RULE VIOLATION COUNT MODEL ON TEST DATA SET.

Test Design	Metal2	Metal3	Metal4	Metal5	Metal6	Metal7	Metal8	Metal9
superblue2	0.2526	0.1211	0.1498	0.0629	0.0310	0.0066	0.0004	0.0018
superblue3	0.4624	0.2817	0.2534	0.1354	0.0627	0.0150	0.0012	0.0039
superblue6	0.5254	0.2782	0.2767	0.1302	0.0568	0.0113	0.0008	0.0026
superblue7	0.6665	0.3632	0.3819	0.1667	0.0585	0.0144	0.0093	0.0033
superblue12	0.6163	0.6733	0.6905	0.3901	0.1579	0.0248	0.0025	0.0049
superblue14	0.5306	0.2776	0.3169	0.1463	0.0652	0.0109	0.0007	0.0022

TABLE VI. RESULTS OF GLOBAL ROUTING AND DETAILED ROUTING USING DIFFERENT CONGESTION MODELS.

Stage	Routing Flow using Model in [11]									Routing Flow using New Model								
	Global Routing Stage					Detailed Routing Stage				Global Routing Stage					Detailed Routing Stage			
Circuit	OF	WL	Via	CPU	Mem	DRC	WL	Via	CPU	DP	WL	Via	CPU	Mem	DRC	WL	Via	CPU
superblue2	2056	9.92	7.51	39.5	18	713	10.78	11.52	271.5	453	9.93	7.62	49.7	22	501	10.72	11.01	170.3
superblue3	2408	5.75	7.13	16.1	13	602	6.24	10.91	242.6	367	5.76	7.25	20.1	14	391	6.19	10.33	152.1
superblue6	1516	5.49	7.27	17.9	13	513	5.93	9.12	179.3	295	5.51	7.36	21.5	15	337	5.81	8.96	116.6
superblue7	994	7.25	10.85	9.1	16	305	7.92	14.32	157.1	169	7.27	11.02	11.6	17	212	7.86	13.92	101.3
superblue9	330	4.34	6.19	6.5	10	116	4.77	9.53	139.0	65	4.35	6.31	8.5	12	79	4.62	9.36	86.5
superblue11	4708	5.51	6.60	11.8	16	1256	6.01	9.91	168.1	797	5.52	6.82	15.1	19	862	5.93	9.80	111.5
superblue12	1302	6.10	10.57	7.6	13	319	6.62	14.00	145.7	171	6.12	10.76	9.9	16	197	6.41	13.62	86.5
superblue14	10716	3.69	4.61	9.2	10	2294	4.01	7.50	170.1	1218	3.72	4.67	11.6	11	1431	3.91	7.37	106.7
superblue19	9407	2.42	3.51	5.9	8	2198	2.64	5.69	136.7	1321	2.46	3.61	7.9	9	1363	2.56	5.61	81.6
Average	n/a	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	n/a	1.01	1.02	1.27	1.15	0.66	0.98	0.97	0.63

TABLE VII. RESULTS OF GLOBAL ROUTING (WITHOUT POST-PROCESSING STAGE) AND DETAILED ROUTING USING DIFFERENT CONGESTION MODELS.

Stage	Routing Flow using Model in [11]									Routing Flow using New Model								
	Global Routing Stage					Detailed Routing Stage				Global Routing Stage					Detailed Routing Stage			
Circuit	OF	WL	Via	CPU	Mem	DRC	WL	Via	CPU	DP	WL	Via	CPU	Mem	DRC	WL	Via	CPU
superblue2	19743	9.93	7.49	31.2	18	523	10.85	14.25	444.0	426	10.00	7.56	39.9	22	333	10.78	12.49	332.4
superblue3	10709	5.71	7.18	11.6	13	523	6.37	13.23	380.2	304	5.78	7.33	12.6	14	328	6.31	11.66	272.2
superblue6	11825	5.42	7.31	10.5	13	496	6.07	13.83	378.8	310	5.48	7.36	12.0	15	312	6.00	12.04	277.7
superblue7	15515	7.16	10.88	7.3	16	655	8.13	21.18	530.5	443	7.27	10.97	8.4	17	389	8.00	17.80	328.1
superblue9	7236	4.30	6.18	4.1	10	383	4.86	11.92	309.7	256	4.36	6.20	4.8	12	221	4.78	10.09	195.9
superblue11	23139	5.44	6.57	8.6	16	369	6.07	12.45	326.3	290	5.50	6.60	10.6	19	217	6.00	10.64	215.8
superblue12	2751	6.07	10.61	6.3	13	1154	6.98	21.09	733.4	541	6.17	10.75	7.3	16	758	6.88	18.71	534.6
superblue14	11049	3.63	4.61	6.7	10	344	4.06	8.64	237.0	214	3.67	4.63	7.6	11	224	4.01	7.52	175.8
superblue19	9575	2.38	3.51	4.5	8	297	2.68	6.62	160.6	149	2.40	3.53	5.5	9	218	2.64	5.70	119.2
Average	n/a	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	n/a	1.01	1.01	1.18	1.15	0.63	0.99	0.87	0.70

- [2] C. J. Alpert, Z. Li, M. D. Moffitt, G.-J. Nam, J. A. Roy, and G. E. T  llez. What makes a design difficult to route. In *ISPD*, pages 7–12, 2010.
- [3] D. Ding, B. Yu, J. Ghosh, and D. Z. Pan. EPIC: Efficient prediction of IC manufacturing hotspots with a unified meta-classification formulation. In *ASP-DAC*, pages 263–270, 2012.
- [4] J. H. Friedman. Multivariate adaptive regression splines. *The annals of statistics*, pages 1–67, 1991.
- [5] R. S. Ghaida, Y. Badr, M. Gupta, N. Jin, and P. Gupta. Comprehensive die-level assessment of design rules and layouts. In *ASP-DAC*, pages 61–66, 2014.
- [6] C.-H. Hsu, H.-Y. Chen, and Y.-W. Chang. Multi-layer global routing considering via and wire capacities. In *ICCAD*, pages 350–355, 2008.
- [7] K. Jeong, A. B. Kahng, B. Lin, and K. Samadi. Accurate machine-learning-based on-chip router modeling. *Embedded Systems Letters*, 2(3):62–66, 2010.
- [8] M.-C. Kim, J. Hu, D. Lee, and I. L. Markov. A SimPLR method for routability-driven placement. In *ICCAD*, pages 67–73, 2011.
- [9] Z. Li, C. J. Alpert, G.-J. Nam, C. C. N. Sze, N. Viswanathan, and N. Y. Zhou. Guiding a physical design closure system to produce easier-to-route designs with more predictable timing. In *DAC*, pages 465–470, 2012.
- [10] M. Pan and C. C. N. Chu. FastRoute: a step to integrate global routing into placement. In *ICCAD*, pages 464–471, 2006.
- [11] Z. Qi, Y. Cai, Q. Zhou, Z. Li, and M. Chen. VFGR: A very fast parallel global router with accurate congestion modeling. In *ASP-DAC*, pages 525–530, 2014.
- [12] J. A. Roy and I. L. Markov. High-performance routing at the nanometer scale. In *ICCAD*, pages 496–502, 2007.
- [13] H. Shojaei, A. Davoodi, and J. T. Linderth. Planning for local net congestion in global routing. In *ISPD*, pages 85–92, 2013.
- [14] N. Viswanathan, C. J. Alpert, C. C. N. Sze, Z. Li, and Y. Wei. The DAC 2012 routability-driven placement contest and benchmark suite. In *DAC*, pages 774–782, 2012.
- [15] S. I. Ward, D. Ding, and D. Z. Pan. PADE: a high-performance placer with automatic datapath extraction and evaluation through high dimensional data learning. In *DAC*, pages 756–761, 2012.
- [16] Y. Wei, C. C. N. Sze, N. Viswanathan, Z. Li, C. J. Alpert, L. N. Reddy, A. D. Huber, G. E. T  llez, D. Keller, and S. S. Sapatnekar. Techniques for scalable and effective routability evaluation. *ACM Trans. Design Autom. Electr. Syst.*, 19(2):17, 2014.
- [17] V. Yutsis, I. Bustany, D. G. Chinnery, J. R. Shinnerl, and W.-H. Liu. ISPD 2014 benchmarks with sub-45nm technology rules for detailed-routing-driven placement. In *ISPD*, pages 161–168, 2014.
- [18] Y. Zhang and C. Chu. GDRouter: interleaved global routing and detailed routing for ultimate routability. In *DAC*, pages 597–602, 2012.